

```

#carregar base
import pandas as pd
from sklearn.model_selection import train_test_split
import sklearn.metrics as metrics

df = pd.read_csv("tech_mental_health_burnout.csv")

#definir X e Y
y = df["burnout_level"]

X = df.drop(
    ["burnout_level", "burnout_score"],
    axis=1
)

#transformar texto em numero
from sklearn.preprocessing import LabelEncoder

for coluna in X.columns:
    if X[coluna].dtype=="object":
        le=LabelEncoder()

        X[coluna]=le.fit_transform(
            X[coluna].astype(str)
        )

le_y=LabelEncoder()
y=le_y.fit_transform(
    y.astype(str)
)

#limpar dados problematicos
import numpy as np

X=X.replace(
    [np.inf, -np.inf],
    np.nan
)

X=X.fillna(
    X.mean()
)

#separar treino e teste

X_treino,X_teste,y_treino,y_teste=(
    train_test_split(
        X,

```

```

y,
test_size=0.2,
random_state=42
)
)

# APLICANDO O ALGORITMO DA ÁRVORE DE DECISÃO #
from sklearn import tree
modeloARV = tree.DecisionTreeClassifier(criterion='entropy',
max_depth=3)
modeloARV = modeloARV.fit(X_treino, y_treino)
predictionsARV = modeloARV.predict(X_teste)

# MOSTRANDO OS RESULTADOS DO ALGORITMO DA ÁRVORE DE DECISÃO #

print("\nMatriz de confusão detalhada da Árvore de Decisão:\n",
      pd.crosstab(y_teste, predictionsARV, rownames=['Real'],
colnames=['Predito'],
      margins=True, margins_name='Todos'))

dic_arv = metrics.classification_report(y_teste, predictionsARV,
      target_names=['High', 'Low', 'Moderate'], output_dict=True)
result_arv = dic_arv['accuracy']
print("Acurácia da Árvore de Decisão: ", result_arv)

```

Matriz de confusão detalhada da Árvore de Decisão:

Predito	1	2	Todos
Real			
0	0	9	9
1	25503	819	26322
2	2017	1652	3669
Todos	27520	2480	30000

Acurácia da Árvore de Decisão: 0.9051666666666667

```

/usr/local/lib/python3.12/dist-packages/sklearn/metrics/
_classification.py:1565: UndefinedMetricWarning: Precision is ill-
defined and being set to 0.0 in labels with no predicted samples. Use
`zero_division` parameter to control this behavior.
    _warn_prf(average, modifier, f"{metric.capitalize()} is",
len(result))
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classificatio
n.py:1565: UndefinedMetricWarning: Precision is ill-defined and being
set to 0.0 in labels with no predicted samples. Use `zero_division`
parameter to control this behavior.
    _warn_prf(average, modifier, f"{metric.capitalize()} is",
len(result))
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classificatio
n.py:1565: UndefinedMetricWarning: Precision is ill-defined and being
set to 0.0 in labels with no predicted samples. Use `zero_division`

```

```

parameter to control this behavior.
_warn_prf(average, modifier, f"{metric.capitalize()} is",
len(result))

# APLICANDO O ALGORITMO KNN #

from sklearn.neighbors import KNeighborsClassifier

modeloKNN = KNeighborsClassifier(n_neighbors=3)
modeloKNN.fit(X_treino, y_treino)
predictionsKNN = modeloKNN.predict(X_teste)

# MOSTRANDO OS RESULTADOS DO ALGORITMO KNN #

print("\nMatriz de confusão detalhada do KNN:\n",
      pd.crosstab(y_teste, predictionsKNN, rownames=['Real'],
                  colnames=['Predito'],
                  margins=True, margins_name='Todos'))

dic_knn = metrics.classification_report(y_teste, predictionsKNN,
                                       target_names=['High', 'Low', 'Moderate'], output_dict=True)
result_knn = dic_knn['accuracy']
print("Acurácia do KNN: ", result_knn)

```

Matriz de confusão detalhada do KNN:

Predito	0	1	2	Todos
Real				
0	1	0	8	9
1	0	25427	895	26322
2	1	2130	1538	3669
Todos	2	27557	2441	30000

Acurácia do KNN: 0.8988666666666667

```

# APLICANDO O ALGORITMO DE REGRESSÃO LOGÍSTICA (Classificador Linear)
#

```

```

from sklearn.linear_model import LogisticRegression
from sklearn import metrics

modelolr = LogisticRegression(max_iter=1000)
modelolr.fit(X_treino, y_treino)

predictionsLR = modelolr.predict(X_teste)

print("\nMatriz de confusão detalhada da Regressão Logística:\n",
      pd.crosstab(y_teste, predictionsLR, rownames=['Real'],
                  colnames=['Predito'],
                  margins=True, margins_name='Todos'))

dic_lr = metrics.classification_report(y_teste, predictionsLR,
                                       target_names=['High', 'Low', 'Moderate'], output_dict=True)

```

```
result_lr = dic_lr['accuracy']
print("Acurácia da Regressão Logística: ", result_lr)
```

Matriz de confusão detalhada da Regressão Logística:

Predito	0	1	2	Todos
Real				
0	1	0	8	9
1	0	25667	655	26322
2	1	1312	2356	3669
Todos	2	26979	3019	30000

Acurácia da Regressão Logística: 0.9341333333333334

```
/usr/local/lib/python3.12/dist-packages/sklearn/linear_model/_logistic.py:465: ConvergenceWarning: lbfgs failed to converge
(status=1):
STOP: TOTAL NO. OF ITERATIONS REACHED LIMIT.
```

Increase the number of iterations (max_iter) or scale the data as shown in:

<https://scikit-learn.org/stable/modules/preprocessing.html>

Please also refer to the documentation for alternative solver options:

https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression

```
n_iter_i = _check_optimize_result(
```

```
# COMITÉ DE CLASSIFICADORES #
```

```
# Armazenam a contagem de votos dos modelos
```

```
tot_alto = 0
```

```
tot_baixo = 0
```

```
# 1. Previsão da Árvore de Decisão
```

```
pred_arv = modeloARV.predict([[35, 1, 2, 5, 3, 1, 40, 2, 3, 0, 4, 4,
3, 7, 2, 6, 2, 7, 0, 3, 2, 1, 0]])
```

```
if pred_arv[0] == 0:
```

```
    tot_alto = tot_alto + 1
```

```
else:
```

```
    tot_baixo = tot_baixo + 1
```

```
print("Previsão Árvore de Decisão:", pred_arv)
```

```
# 2. Previsão do KNN
```

```
pred_knn = modeloKNN.predict([[35, 1, 2, 5, 3, 1, 40, 2, 3, 0, 4, 4,
3, 7, 2, 6, 2, 7, 0, 3, 2, 1, 0]])
```

```
if pred_knn[0] == 0:
```

```
    tot_alto = tot_alto + 1
```

```
else:
```

```
    tot_baixo = tot_baixo + 1
```

```
print("Previsão KNN:", pred_knn)
```

```

# 3. Previsão da Regressão Logística
pred_lr = modelolr.predict([[35, 1, 2, 5, 3, 1, 40, 2, 3, 0, 4, 4, 3,
7, 2, 6, 2, 7, 0, 3, 2, 1, 0]])
if pred_lr[0] == 0:
    tot_alto = tot_alto + 1
else:
    tot_baixo = tot_baixo + 1
print("Previsão Regressão Logística:", pred_lr)

# DECISÃO FINAL DO COMITÊ (Votação da Maioria) #

if tot_baixo >= 2:
    print("\n0 comitê acredita que o profissional apresenta baixo
risco de Burnout.")
else:
    print("\n0 comitê acredita que o profissional apresenta alto risco
de Burnout (Alerta!).")

# Verificação de Unanimidade
if tot_baixo == 3 or tot_alto == 3:
    print("0 comitê foi unânime em sua decisão.")

Previsão Árvore de Decisão: [1]
Previsão KNN: [1]
Previsão Regressão Logística: [1]

0 comitê acredita que o profissional apresenta baixo risco de Burnout.
0 comitê foi unânime em sua decisão.

/usr/local/lib/python3.12/dist-packages/sklearn/utils/
validation.py:2739: UserWarning: X does not have valid feature names,
but DecisionTreeClassifier was fitted with feature names
    warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:27
39: UserWarning: X does not have valid feature names, but
KNeighborsClassifier was fitted with feature names
    warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:27
39: UserWarning: X does not have valid feature names, but
LogisticRegression was fitted with feature names
    warnings.warn(

```